



Efficient Approximate Read Mapping Algorithm for Genomic Data

Bartosz Smolarczyk, Wiktor Grzankowski • 24.11.2024



Problem Statement

Overview:

- Map genomic reads (~1 kbp) to a reference sequence (~20 Mbp)
- Handle up to 10% sequencing error (substitutions, insertions, deletions)

Key requirements:

- Min 80% reads mapped (goal: 99%)
- Max 1% reads mapped incorrectly (goal: 0%)
- Efficient time complexity ($5 + r/10$ minutes) and memory complexity (< 1 GB)



Approach

Approach

Once, given reference sequence

- Build a suffix array for the reference sequence

Then, for each read

- Shingle the read to reduce search complexity
- Use dynamic programming to refine approximate matches

Building the suffix array

- Suffix array enables fast substring search by allowing binary search-like operations on the sorted suffixes
- Karkkainen-Sanders algorithm is used
- Time complexity is $O(n)$, where n is the length of the reference sequence
- Space complexity is $O(n)$, due to the use of auxiliary arrays
- It's created upon Index class initialization

Shingling

What is it?

- Shingling breaks the read into smaller overlapping substrings called shingles
- Used to localize regions in the reference genome for approximate matching

Why use it?

- Reduce search space - focus only on regions where shingles match
- Handles sequencing errors - adding shingles with edit distance = 1 allows for tolerance of small errors

Shingling - implementation

In a loop:

- Compute a set of 0- and 1-edit-distance shingles from the read string.
 - For each shingle, memorize its sequence, offset from the beginning of read sequence and range of estimated occurrence
 - Length and amount of shingles depend on different parameters, such as length of read, expected error rate or step in indexes between neighbouring ones
 - First compute exact shingles from the read
 - Then, for each shingle, compute its 1-neighbourhood

KEditDP - Dynamic Programming for Edit Distance

KEditDP class finds the best match for a read in a reference, allowing substitutions, insertions and deletions

Works as in provided code, with slight change to compute the matrix diagonally.

Querying

For each shingle in the set

- Query suffix array for its position
- Compute its range of estimated occurrence positions in reference
- Add (+1) and (-1) to cumulative occurrence map at range's start and stop positions accordingly

For each entry of (*index*, *count*) in the sorted cumulative occurrence map

- *Count* to cumulative count
- If cumulative count is the greatest so far, remember the *index* as most likely matching occurrence

With the index of most likely read matching occurrence in reference

- Compute the KEdit DP class to find best edit distance and its starting and ending positions
 - Implementation was optimised to compute the matrix diagonally, allowing to take advantage of numpy vectorization



Plotting

Plots

Uncomment the line:

```
reference_substring = self.reference[reference_substring_start : reference_substring_end]

# self.__plot_occurrence_map(occurrence_map, read_id)

kEditDP = KEditDP(pattern, reference_substring, 2 * edist)
```

provides ability to plot the result of shingle occurrence mapping over the reference string.

It's a great way of verifying the script's accuracy and understanding its work principles.



Achieving full accuracy

Full correctness, 99% reads mapped

- In some reads, it is difficult to find good candidates for the result
- While satisfying maximum requirements for the task, it is ok to skip 1/100 mappings
- It's done by sorting the results by best edit distance and leaving the worst at most 1%

```
results = [index.query(str(read.seq), int(len(read.seq) * ERROR_RATE), read.id) for read in reads]
results.sort(key=lambda result: result.best_edist)

reliable_results = results[: math.ceil(len(results) * READ_RELIABLE_THRESHOLD)]
reliable_results.sort(key=lambda result: result.read_id)
```



Testing

Tests

The solution passed all tests attached to the task, while achieving all requirements, including

- No incorrectly mapped reads
- 99% reads mapped
- Mapping time of less than 1 second per read



Complexity

Complexity - notation

P	Length of searched pattern (read)
R	Length of reference string
S	Length of one shingle
N	Number of shingles generated per pattern (read)
D	Distance between shingles' starting positions
E	Chosen error rate

Complexity - estimates

The following estimates hold for the presented implementation:

- $S \approx 2 / E$
- $N \approx P * S * 6 / D$
 - 6 comes from generating 1-neighbourhood. For one shingle of length S there are 6 possible operations at each position
 - 1 substitution
 - 1 deletion
 - 4 insertions

Complexity - key factors

- $O(R)$ - initialising the index. Has to be done once for the given reference string
- $O(S + \log(R))$ - querying the suffix array for occurrences of one shingle (amortised). For it to be a fixed complexity we would have to sacrifice memory for storing LCP arrays, but we cannot do that with existing memory limit
- $O(N * \log(N))$ - sorting the occurrence range map
- $O(P^2)$ - computing the KEditDP

This allows us to finally write down the complexity for processing a single pattern, assuming Index exists:

- $O(N * (S + \log(R)) + N * \log(N) + P^2) = O(N * (S + \log(R * N)) + P^2)$

Complexity - final remark

With the complexity being

- $O(N * (S + \log(R * N)) + P^2)$

If we were to substitute the values from the implementation, that is

- $S \approx 20, N = 1.3 * 10^4$
- Assuming maximum reference length $R = 2 * 10^7$

We can see that

- $O(N * (S + \log(R * N))) \approx 8 * 10^6$
- $O(P^2) \approx 10^6$

Which means that we have implemented the solution choosing the right values of S and N to achieve high accuracy without exceeding the unavoidable computational overhead of $O(P^2)$